

Applied A.I. Solutions Development Program
DEEP LEARNING I

Term Project

RBC AI Banking Agent

GROUP 7

Bikash Giri (101575097)
Chia-Wei Chang (101570243)
Diparshan Bhattarai (101494737)
Hsi-Teng Liu (101576074)
Gavriel Kirichenko (101119609)
Abdul-Rasaq Omisesan (101571156)
Callum Arul (101585383)
Friba Hussainyar (101591222)

DEMO

<https://github.com/frogcoder/llm-chatbot/tree/main>

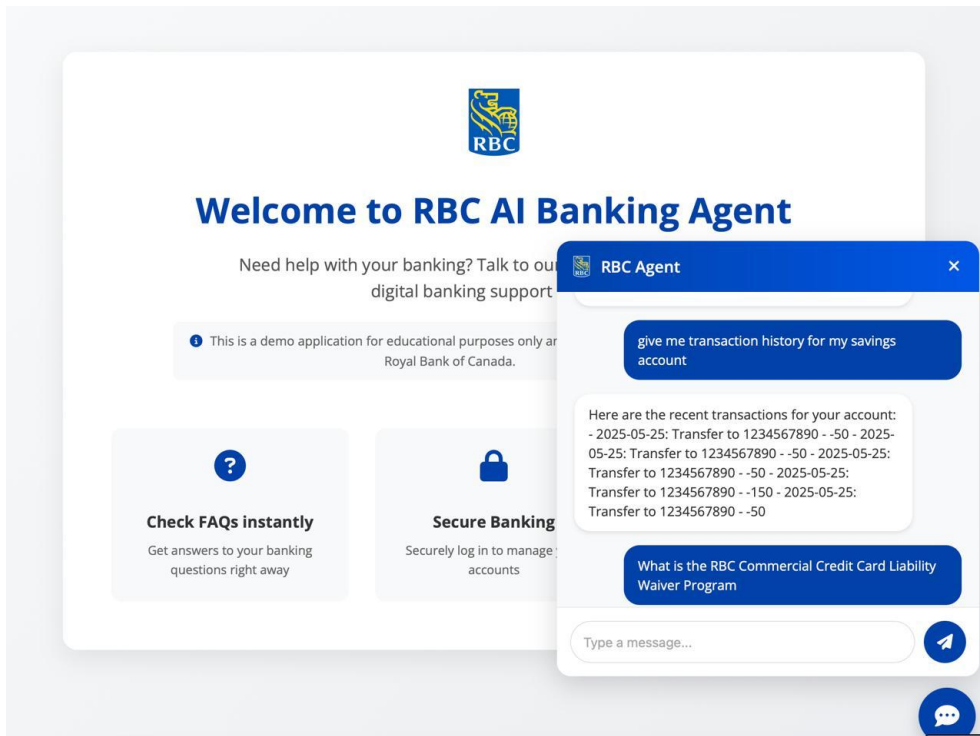
Agenda

- Demo: Callum Arul
- Introduction and Workflow Architecture: Hsi-Teng Liu
- Core Functionality:
 - Frontend/UI: Bikash Giri
 - Backend/API
 - Authentication API: Abdul-Rasaq Omisesan
 - Bank API for Transfer: Hsi-Teng Liu
 - RAG System: Diparshan Bhattarai & Callum Arul
 - Web Scraper: Friiba Hussainyar
 - MCP: Gavriel Kirichenko & Chia-Wei Chang

Introduction

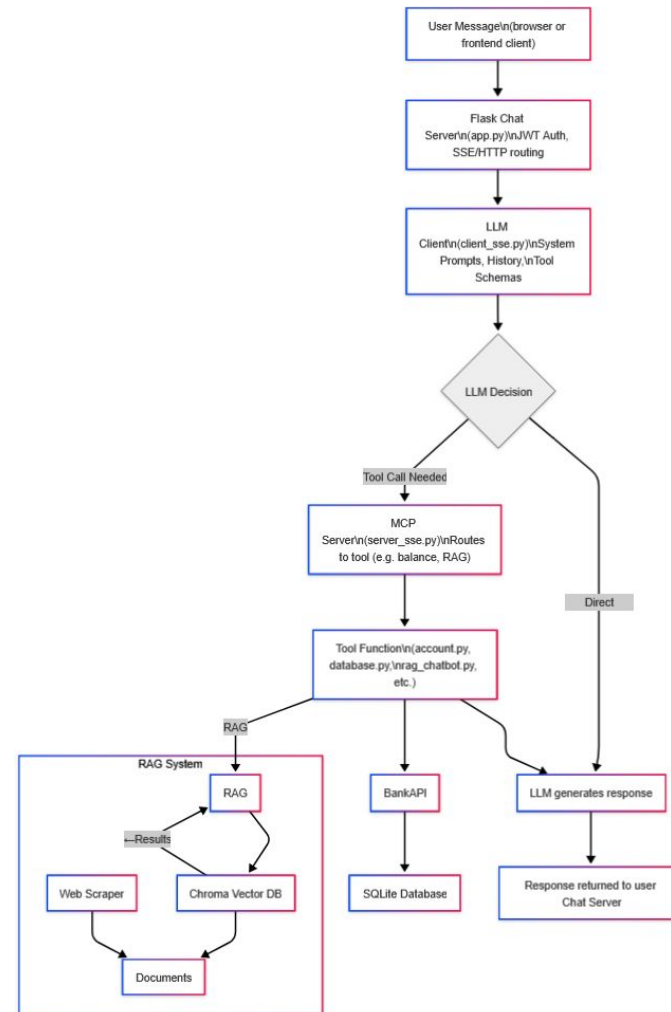
Components

1. RAG System
2. MCP System
3. Banking Operations
4. Conversation Management
5. Web Interface
6. SQLite database for account and transaction management



Introduction

Architecture & Data Flow



Frontend/UI

- Developed a **modern chatbot interface** using HTML, CSS, and JavaScript
- Integrated with Flask backend using **RESTful API calls** (`/chat`, `/auth`)
- Built a **floating chatbot** UI that mimics real-world banking assistants (e.g., RBC)
- Styled UI to match **professional banking design**, including:
 - > Animated robot icon
 - > Chat bubble-style layout
 - > Responsive layout for desktop/laptop use

Component

Interaction

`chat.html`

Loads UI with robot icon

`script.js`

Sends messages to Flask (`/chat`)

Flask '`/chat`'

Routes user input to LLM backend

Session
handling

Backend uses Flask `session` to store login state

Front-End Authentication Overview

Why authentication matters

- Security & Privacy
- Personalization & Context
- Data Integrity
- Compliance & Auditing

Key Technologies

- HTML
 - CSS
 - Java Script
- Fetch API, DOM APIs, Event Listeners, SSE

High-Level User Flow

- User clicks "Chat" → toggleChat()
- If no token → show Login Modal
- User submits credentials → submitLogin() → POST /auth/login
- Store Access Token → hide modal → show success message
- Send/receive chat → every /chat request includes Authorization: Bearer <token>

Login Triggering

```
// Toggle the visibility of the chat popup
function toggleChat() {
  const chatPopup = document.getElementById('chat-popup');
  chatPopup.classList.toggle('hidden');

  // If we're opening the chat and not logged in, show login modal
  if (!chatPopup.classList.contains('hidden') && !accessToken) {
    document.getElementById('login-modal').classList.remove('hidden');
    // Auto-focus on the username field
    document.getElementById('username').focus();
  }
}
```

Submitting Credentials

```
// Handle user login submission
async function submitLogin() {
  const username = document.getElementById('username').value;
  const password = document.getElementById('password').value;

  try {
    const res = await fetch('/auth/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username, password })
    });
    const data = await res.json();

    if (res.ok && data.status === 'success') {
      accessToken = data.access_token;
      document.getElementById('login-modal').classList.add('hidden');

      const chatBox = document.getElementById('chat-box');
      const msgElem = document.createElement('div');
      msgElem.classList.add('message', 'success-message');
      msgElem.innerHTML = '✅ Login successful! You can now proceed.';
      chatBox.appendChild(msgElem);

      // Add welcome message after successful login
      setTimeout(() => {
        appendMessage('Bot', 'Welcome to RBC Banking Assistant! How can I help you today?');
        // Focus on the message input field after welcome message appears
        document.getElementById('message').focus();
      }, 500);

      chatBox.scrollTop = chatBox.scrollHeight;
    } else {
      appendMessage('System', '❌ Login failed. Try again.');
    }
  } catch (err) {
    console.error('Login error:', err);
    appendMessage('System', '⚠️ An error occurred. Please try again.');
  }
}
```

Authorization Header

```
// Send a chat message to the backend
async function sendMessage() {
  const input = document.getElementById('message');
  const message = input.value.trim();
  if (!message) return;
  input.value = '';
  appendMessage('You', message);

  // We should already be logged in at this point
  if (!accessToken) {
    appendMessage('System', '🔒 Session expired. Please login again.');
    document.getElementById('login-modal').classList.remove('hidden');
    return;
  }

  try {
    // Show typing indicator
    showTypingIndicator();
  }
```

```
const res = await fetch('/chat', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  },
  body: JSON.stringify({ message })
});

const data = await res.json();

// Remove typing indicator after we get the response
removeTypingIndicator();

// Display the bot's response
appendMessage('Bot', data.reply);
} catch (err) {
  // Remove typing indicator
  removeTypingIndicator();
}
```

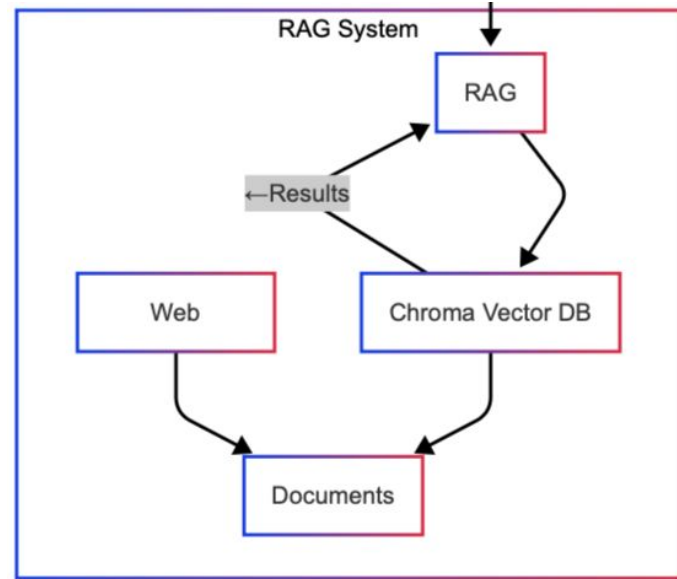
Backend Bank API

- list_accounts (user_id)
- list_transfer_target_accounts (from_account, user_id)
- transfer_between_accounts (user_id, from_account, to_account, amount, description)
- list_transactions (user_id, account, days)

The “**account**” parameter (from_account, to_account) can be the **number** or **name** of the account for the chatbot could either extract account number or name from messages.

RAG Chatbot - Architecture

- Document processing pipeline
- Vector database for semantic search
- LLM for generating contextual responses
- Integration with banking assistant



Web Scraper

```
def explore_page(self, url):  
    """Explore a page for links and documents"""  
    if url in self.visited_urls:  
        return  
  
    self.visited_urls.add(url)  
    print(f"Exploring: {url}")  
  
    try:  
        # Respect the website by waiting between requests  
        time.sleep(self.delay)  
  
        # Get the page content  
        response = requests.get(url, headers=self.headers, timeout=10)  
        response.raise_for_status()  
  
        # Parse HTML  
        soup = BeautifulSoup(response.text, 'html.parser')  
  
        # Check if page is relevant  
        is_relevant = self.is_relevant_page(url, soup)  
  
        # Find all links  
        for link in soup.find_all('a', href=True):  
            href = link.get('href')  
            full_url = urljoin(url, href)  
  
            # Check if it's a document  
            if any(href.lower().endswith(ext) for ext in ['.pdf', '.docx', '.doc', '.xlsx', '.xls']):  
                self.download_document(full_url)
```

Embedding and storing in Vector DB

```
def load_documents(directory_path):  
    # Get all PDF files in the directory  
    pdf_files = glob.glob(os.path.join(directory_path, "**/*.pdf"), recursive=True)  
  
    # Get all text files in the directory  
    txt_files = glob.glob(os.path.join(directory_path, "**/*.txt"), recursive=True)  
  
    # Process files and return documents  
    # ...
```

```
def split_documents(documents, chunk_size=1000, chunk_overlap=200):  
    """Split documents into chunks for better processing"""  
    text_splitter = RecursiveCharacterTextSplitter(  
        chunk_size=chunk_size,  
        chunk_overlap=chunk_overlap,  
        length_function=len,  
    )  
    chunks = text_splitter.split_documents(documents)  
    print(f"Split into {len(chunks)} chunks")  
    return chunks
```

```
def create_vector_store(documents, persist_directory=None):  
    # Initialize the embeddings using Gemini  
    embeddings = GoogleGenerativeAIEmbeddings(  
        model="models/embedding-001",  
        google_api_key=api_key  
    )  
  
    # Create the vector store  
    vector_store = Chroma.from_documents(  
        documents=documents,  
        embedding=embeddings,  
        persist_directory=persist_directory  
    )
```

Chatbot

```
class RBCChatbot:
    def __init__(self, persist_directory=None):
        # Initialize the vector store
        self.vector_store = load_vector_store(persist_directory)

        # Initialize the LLM
        self.llm = ChatGoogleGenerativeAI(model="gemini-1.5-pro", temperature=0.2)

        # Create the retrieval chain
        self.qa_chain = RetrievalQA.from_chain_type(
            llm=self.llm,
            chain_type="stuff",
            retriever=self.vector_store.as_retriever(search_kwargs={"k": 5}),
            return_source_documents=True
        )
```

Integration

```
def answer_question(self, question):
    # Get the answer from the chain
    result = self.qa_chain.invoke({"query": enhanced_prompt})

    # Extract the answer and sources
    answer = result["result"]
    source_docs = result["source_documents"]

    # Format sources for citation
    sources = []
    for doc in source_docs:
        if hasattr(doc, "metadata") and "source" in doc.metadata:
            sources.append(doc.metadata["source"])

    # Return the answer and unique sources
    return {
        "answer": answer,
        "sources": list(set(sources))
    }
```

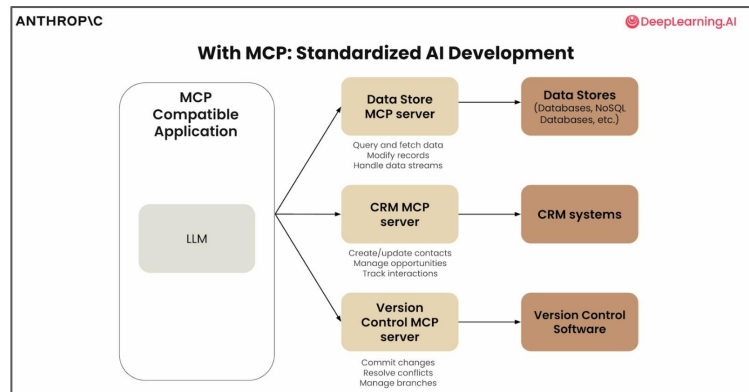
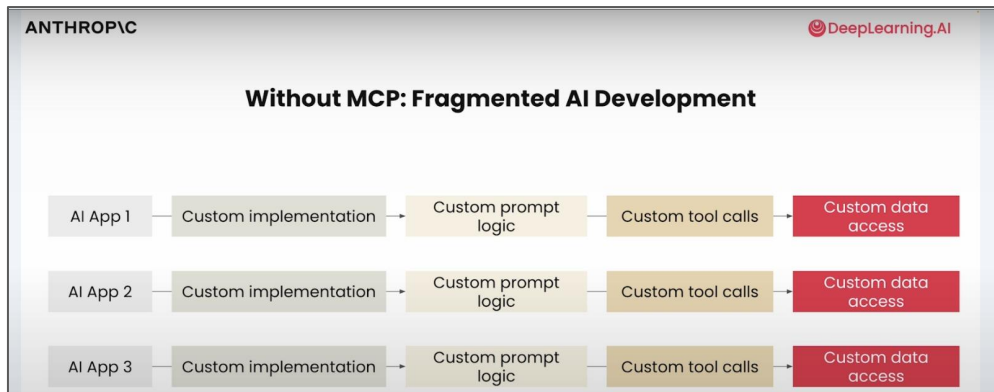
```
@mcp.tool()
def answer_banking_question(question: str) -> dict:
    """
    Answer a banking question using the RAG system with RBC documentation.
    Only for banking, financial services, or RBC-related questions.
    Returns the answer and sources.
    """
    print(f"[RAG] Processing question: {question}")

    # Process the question
    result = chatbot.answer_question(question)

    return {
        "answer": result["answer"],
        "sources": result["sources"]
    }
```


What is MCP?

- MCP is a modular framework for building and hosting microservice tools.
- It enables communication between components via defined tools/APIs.
- Supports asynchronous communication with tools callable via sessions.
- Facilitates scalable and interactive applications by combining AI, APIs, and business logic.



MCP in Our Interactive Banking Assistant

MCP acts as the backend server exposing banking-related tools:

- Listing user accounts
- Getting account balances
- Retrieving transaction history
- Transferring funds between accounts
- Answering banking questions using RAG (Retrieval-Augmented Generation) system

Tools are decorated with `@mcp.tool()` and callable asynchronously:

Sample code of the MCP Client and Server

MCP Client

```
import os, asyncio
from mcp import ClientSession
from mcp.client.sse import sse_client
from google import genai
from dotenv import load_dotenv

load_dotenv("../.env")
client = genai.Client(api_key=os.getenv("GEMINI_API_KEY"))

async def main():
    async with sse_client("http://127.0.0.1:8050/sse") as (read_stream, write_stream):
        async with ClientSession(read_stream, write_stream) as session:
            await session.initialize()

            prompt = """
            You are my bank assistant. Help me manage my accounts.
            What accounts does user_abc123 have?
            Can I transfer from Savings to Checking?
            Transfer 100.50 from account ABC123 to
            """

            response = await client.aio.models.generate_content(
                model="gemini-1.5-pro",
                contents=prompt,
                config=genai.types.GenerateContentConfig(
                    temperature=0,
                    tools=[session],
                ),
            )
            print("\n📧 Gemini response:")
            print(response.text)
```

Connect MCP Client

Tell LLM the tools we have

MCP Server

```
from mcp.server.fastmcp import FastMCP
from dotenv import load_dotenv
from decimal import Decimal

# Load environment variables from .env file
load_dotenv("../.env")

# Create the MCP server
mcp = FastMCP(name="BankMCP", host="127.0.0.1", port=8050)

# Tool 1: List all accounts belonging to a user
@mcp.tool()
def list_user_accounts(user_id: str) -> list[dict]:
    """List all accounts for a given user."""
    accounts = list_accounts(user_id)
    print(f"[DEBUG] list_user_accounts called with user_id={user_id}")
    print(f"[DEBUG] Accounts: {accounts}")
    return [account.__dict__ for account in accounts]

print(f"[DEBUG] list_target_accounts called with user_id={user_id}, from_account={from_account}")
print(f"[DEBUG] Transfer targets: {accounts}")
return [account.__dict__ for account in accounts]
```

Set MCP Host

MCP Decorators

The MCP client reads the available tools based on the descriptions provided in the comments, as well as the input and output parameters.

Thank You